

MEDICAL STORE MANAGEMENT SYSTEM

NAME:KIMAIYO TRIZAH JEBET
ADMISSION NUMBER:BSCCS/2024/53193
UNIT CODE:BIT3208
UNIT NAME:ADVANCED WEB DESIGN AND DEVELOPMENT
LECTURER:MICHAEL NYORO
PROJECT TITLE:MEDICAL STORE MANAGEMENT SYSTEM

WEEK1;Environment setup

TOOLS USED

Vs code
Xampp
Node.jsChrome

Insallaation of node.js

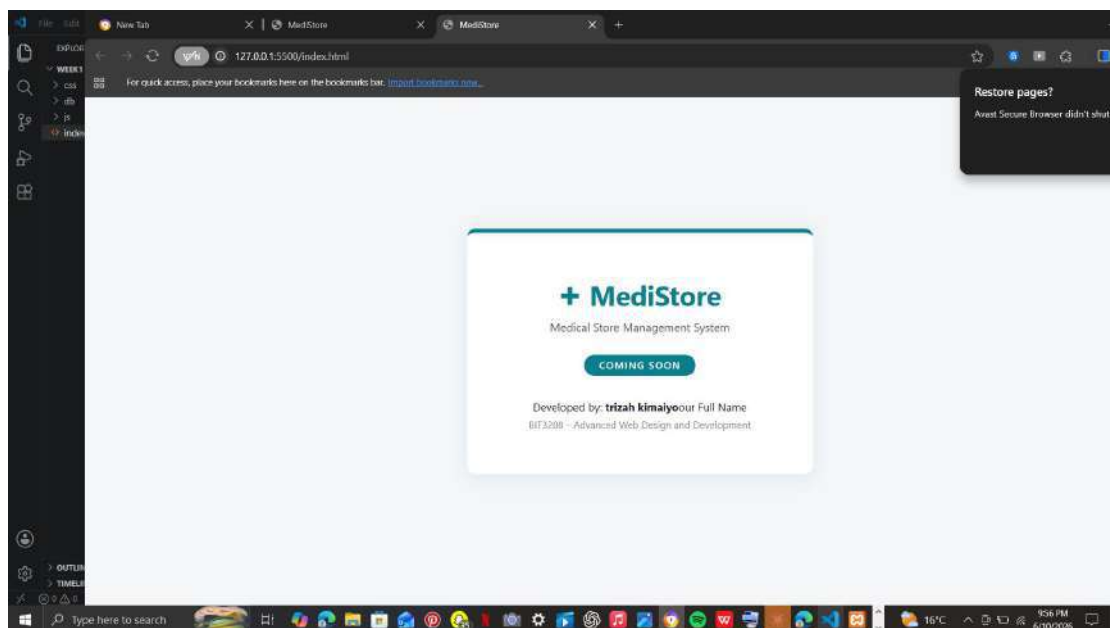
```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>node -v
v20.16.0

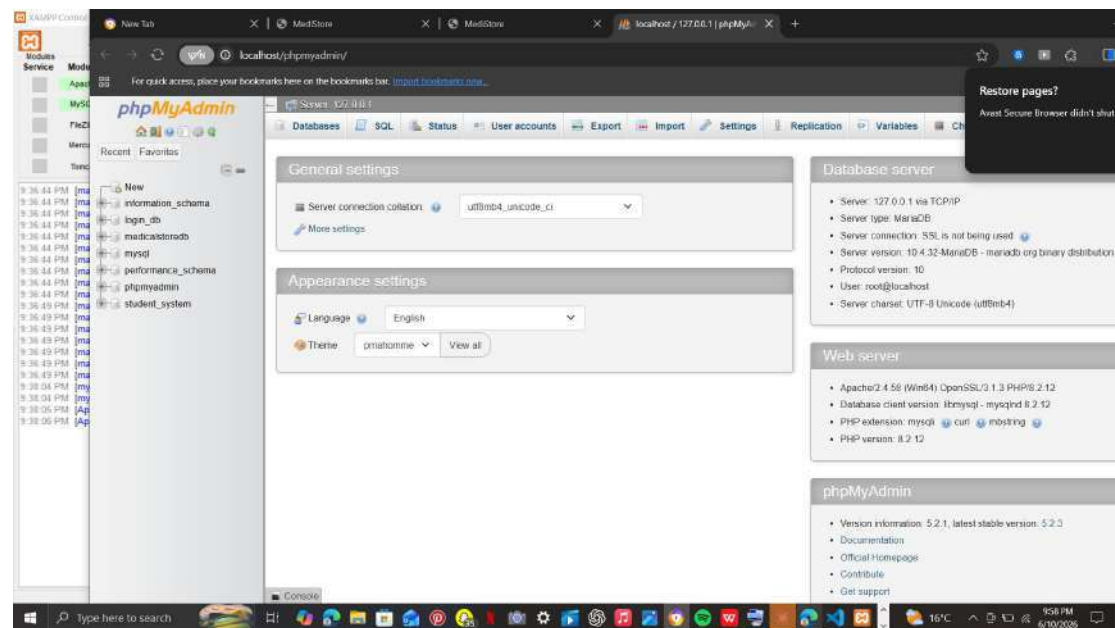
C:\Users\user>npm -v
11.13.0

C:\Users\user>
```

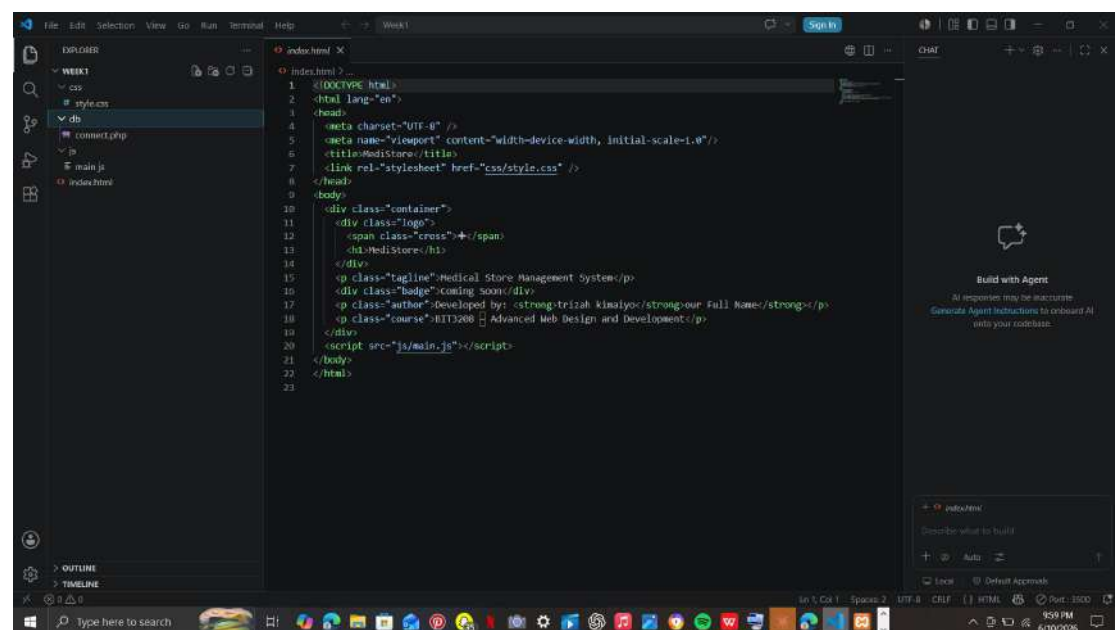
Localhost running



Database conection success



Project folder structure in vscode

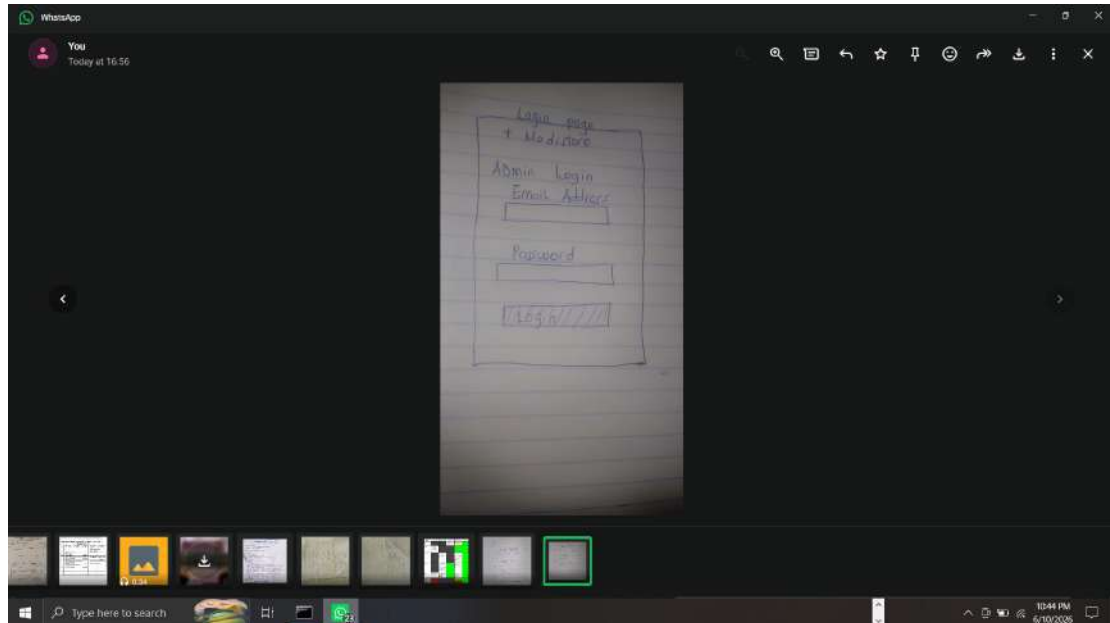


Student reflection

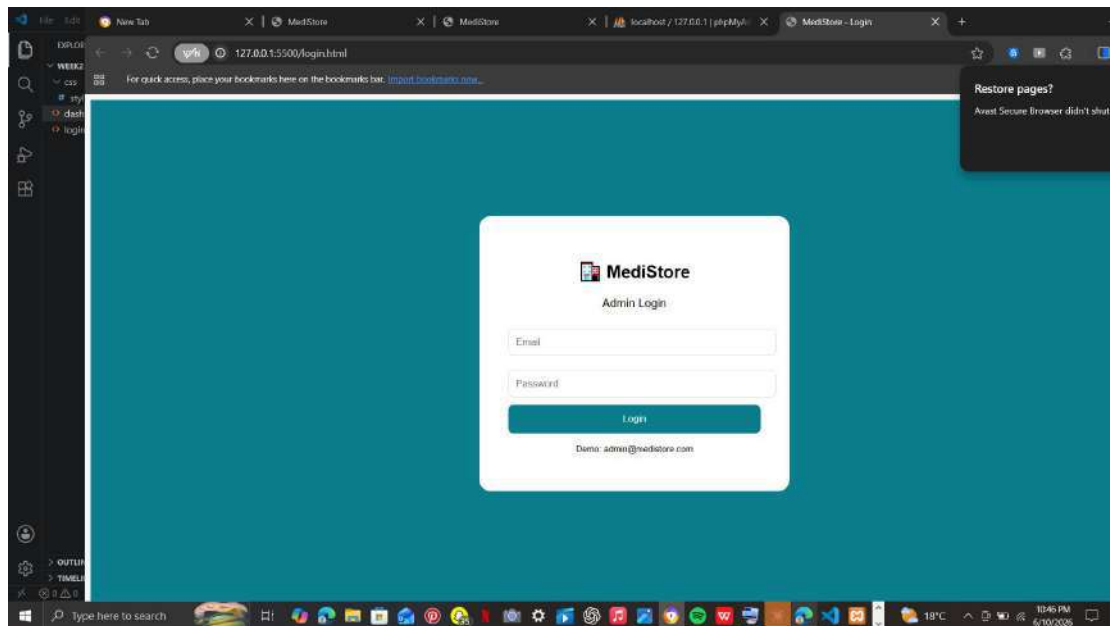
I successfully set up my development environment for the medistore project. I installed xampp and verified that everything works by testing localhost and creating database. The setup was smooth and I learned how web servers and database communicate locally.

WEEK 2.PERSONAL PORTFOLIO

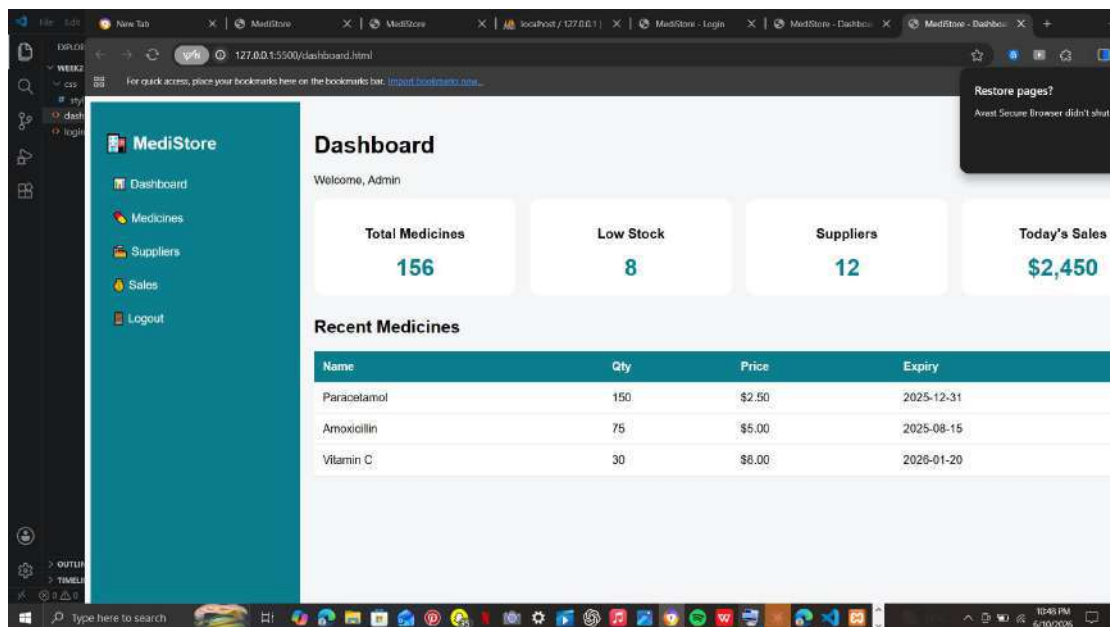
1. Hand drwn ireframes



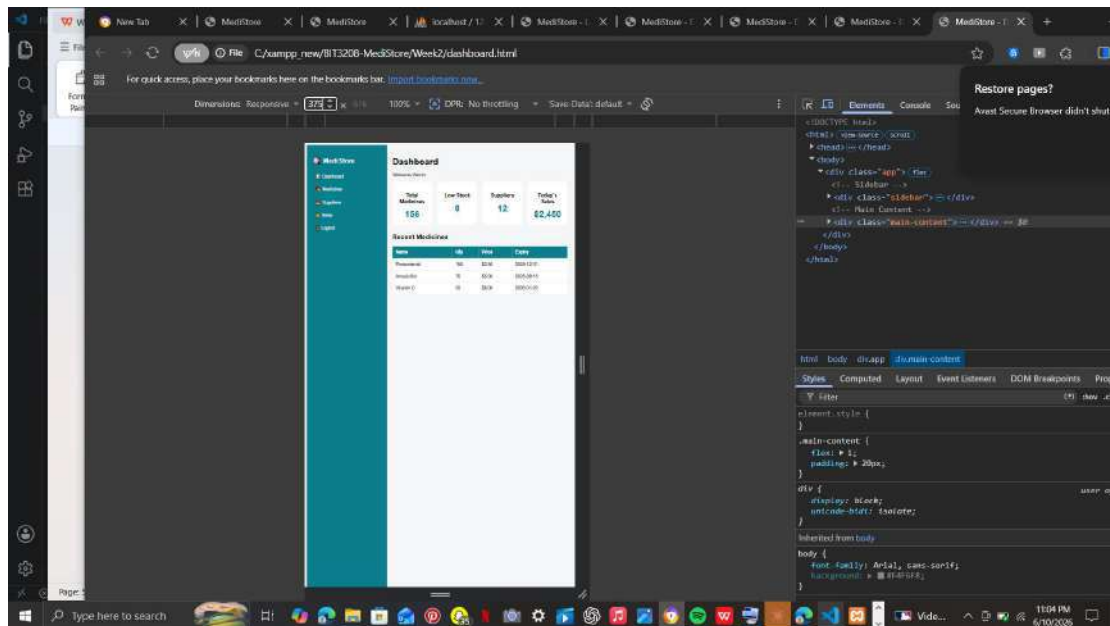
2. Digital GUI login.html



3. Digital GUI dashboard.html



4. Mobie mockup

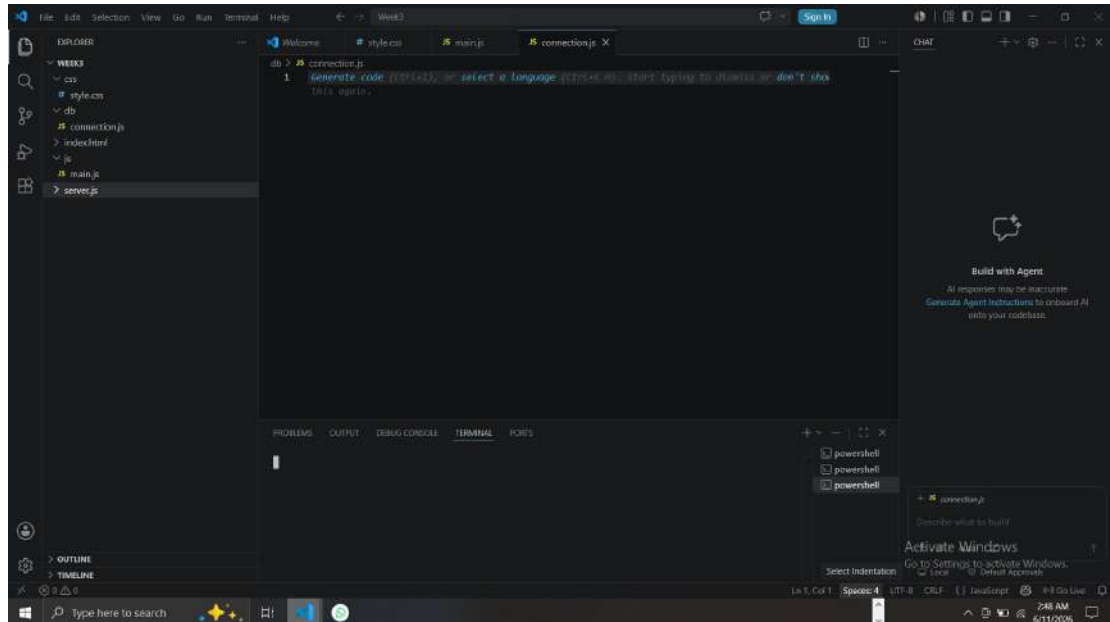


Student reflection

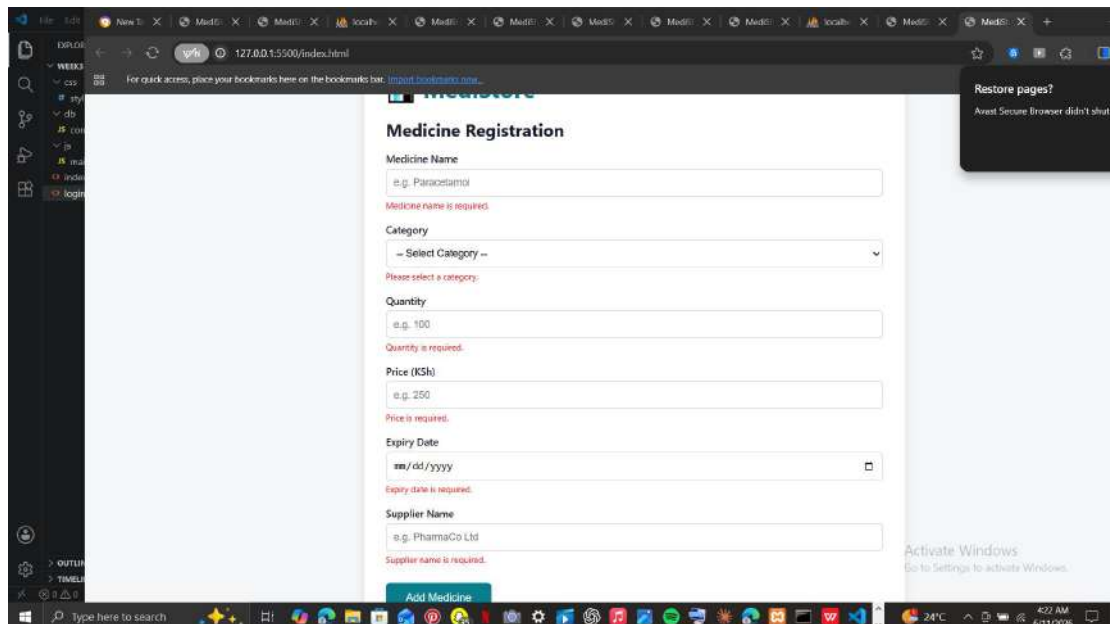
During Week 2, I designed the complete visual layout for my MediStore system. I created hand-drawn wireframes on paper for the login page, dashboard, medicine list, and add form. I then built HTML/CSS mockups in vscode. I also tested the mobile responsiveness by using Chrome DevTools at 375px width, which stacked the stat cards vertically. I learned that planning the design before coding makes development much

Week 3:form validation system

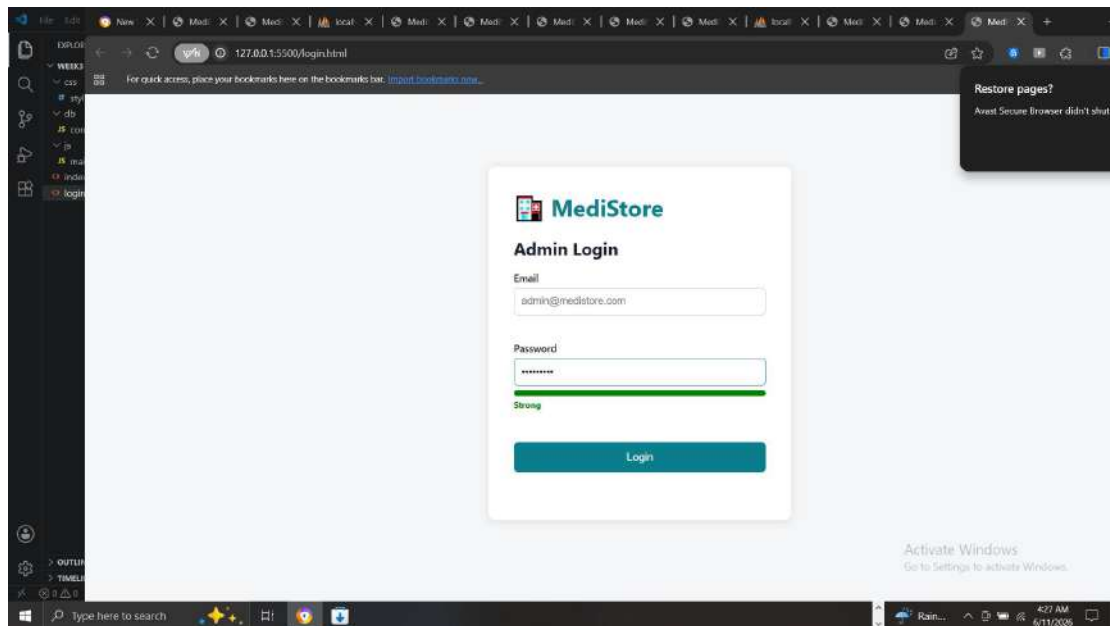
3.1 FOLDER STRUCTURE



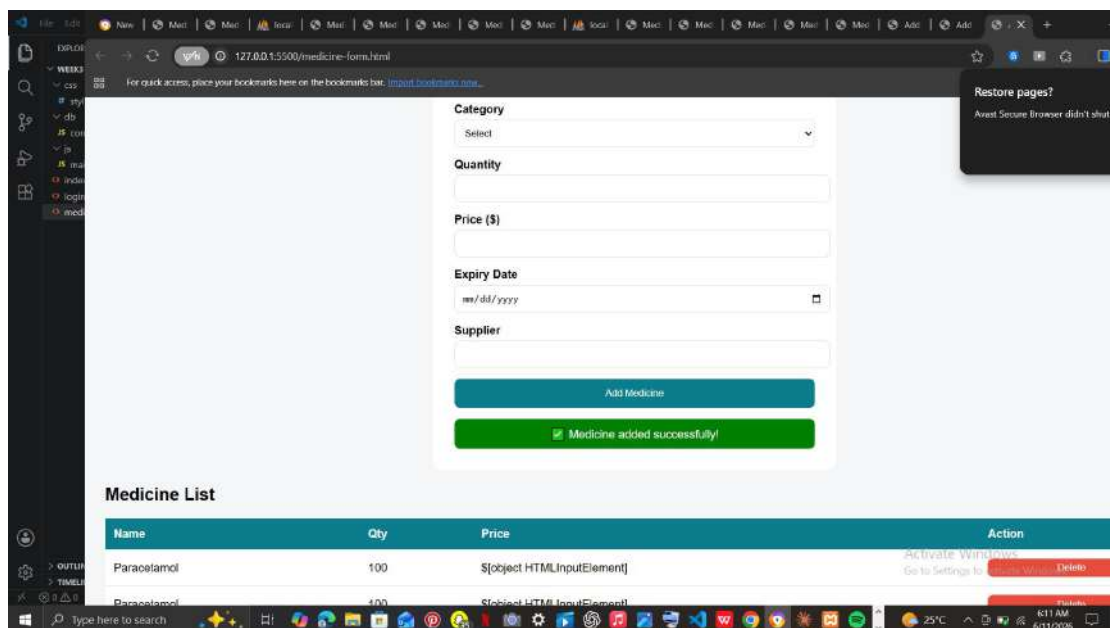
3.2 VALIDATION ERRORS



3.3 PASSWORD STRENGTH



3.4 MEDICINE IN TABLE



Student reflection

During Week 3, I built the frontend functionality for my MediStore system using JavaScript. I created a medicine registration form with validation, a password strength checker on the login page, and a dynamic table that displays added medicines using DOM manipulation.

I learned how to use JavaScript to validate form inputs before submission. I implemented checks for empty fields, positive numbers for quantity and price, and future dates for expiry. I also learned how

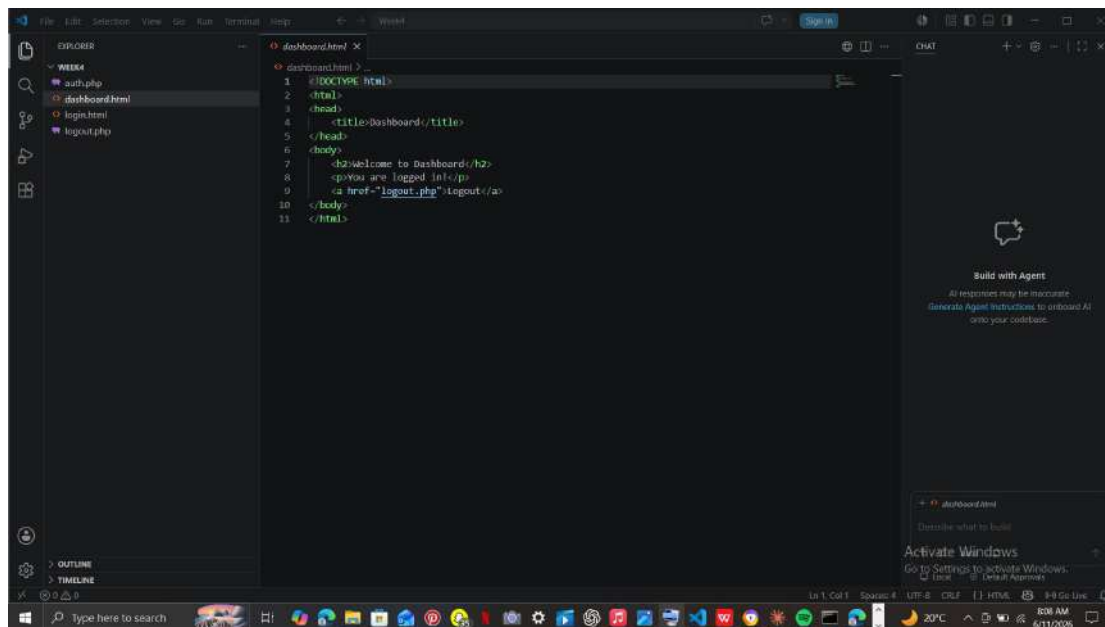
to create a password strength meter that changes color from red (weak) to green (strong) based on password complexity.

The main challenge was getting the form validation to work correctly with all fields and displaying error messages under each field. I also had trouble with the DOM manipulation when trying to add new medicines to the table. The JavaScript errors in the console took time to debug.

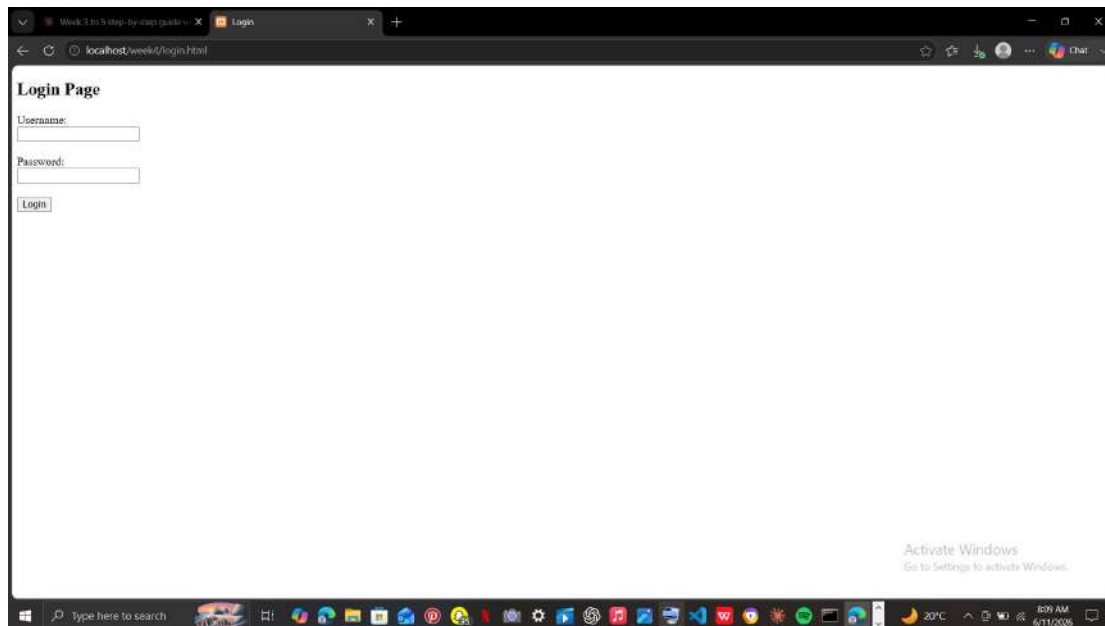
I fixed the validation by checking each field one by one and displaying specific error messages. For the table, I used an array to store medicines and a function to update the table whenever a new medicine was added. I learned to use `preventDefault()` to stop the form from refreshing the page.

WEEK 4.SERVER SIDE COMPONENTS

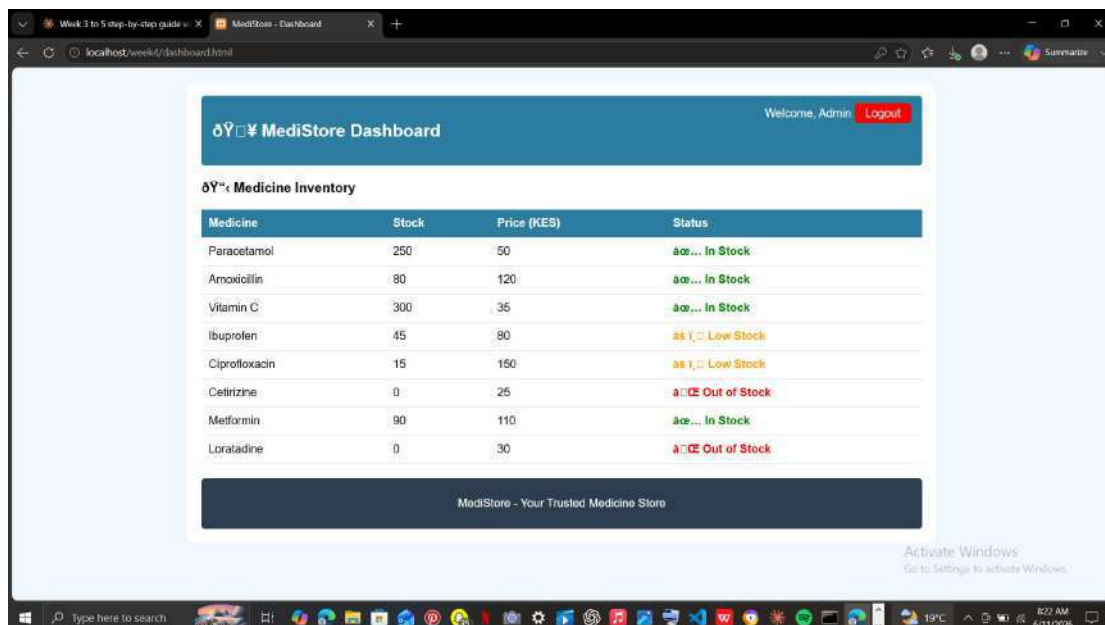
4.1folder structure



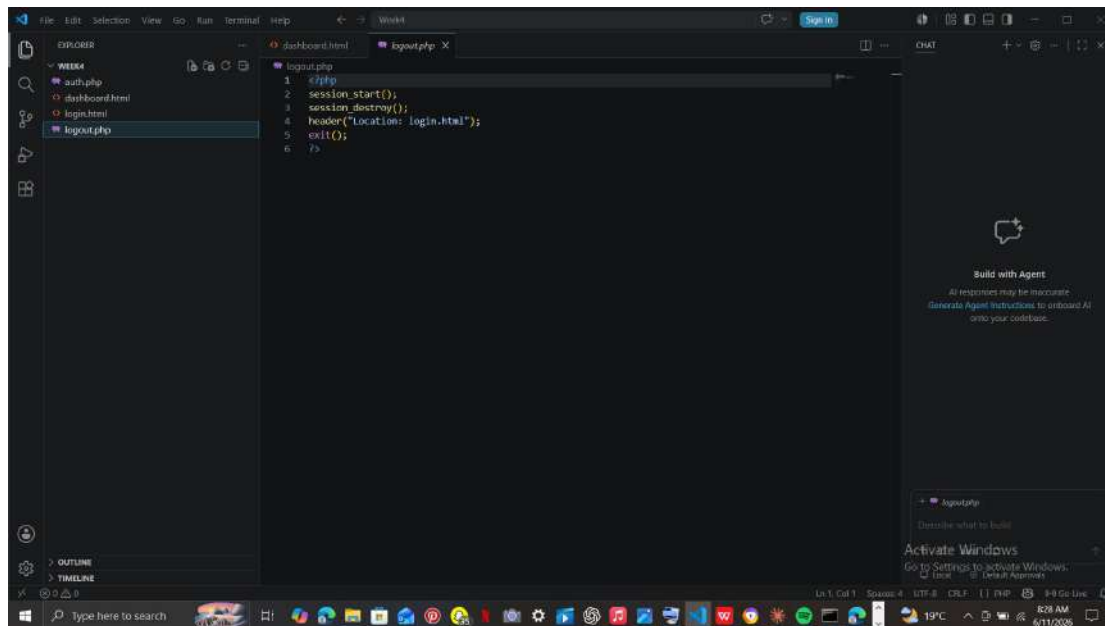
4.2 login page



4.3 medistore dashboard displaying medicine inventory



4.4 logout code

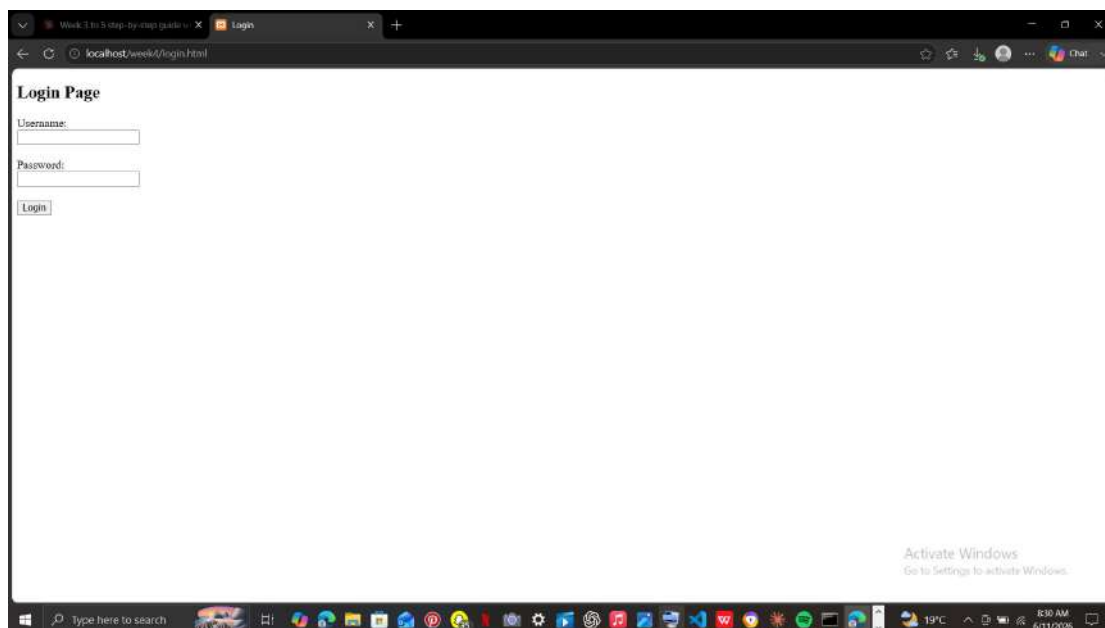


The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure for 'WEEK4' containing 'auth.php', 'dashboard.html', 'login.html', and 'logout.php'. The main editor window is open to 'logout.php', which contains the following PHP code:

```
1 <?php
2 session_start();
3 session_destroy();
4 header("Location: login.html");
5 exit();
6 ?>
```

The right sidebar shows a 'CHAT' panel with a 'Build with Agent' section and an 'Activate Windows' watermark at the bottom.

4.5 afterlogout



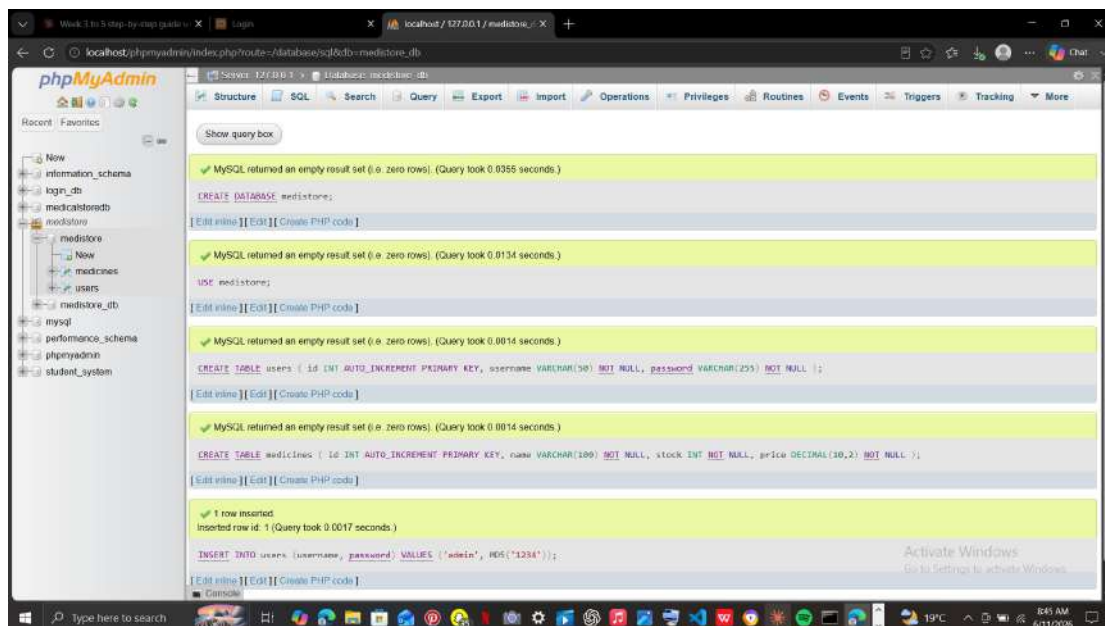
STUDENT REFLECTION

During Week 4, I built a complete login and logout system for the MediStore project by creating four files: login.html for the login form, auth.php to check the username and password, dashboard.html to display the medicine inventory after successful login, and logout.php to destroy the session and redirect to the login page. I faced several challenges along the way, including a redirect loop bug where logout.php was redirecting to itself instead of login.html, a 404 "Not Found" error because my

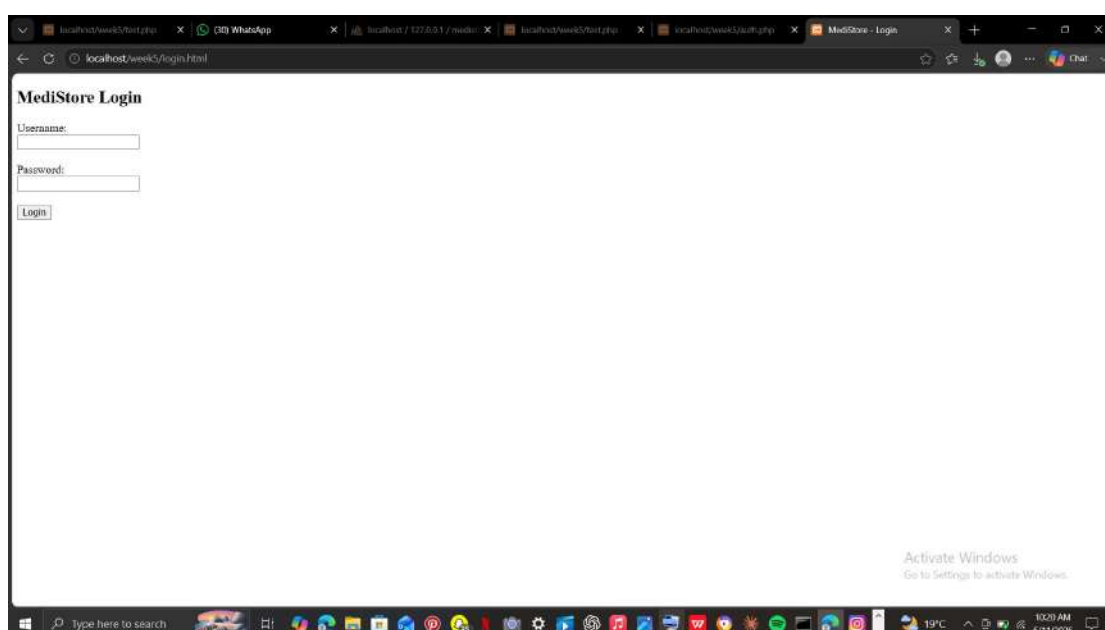
week4 folder was not inside the xampp htdocs directory, . I also learned that dashboard.html can still be accessed directly without logging in because HTML files do not check sessions, which is something I would improve in the future by converting it to PHP. Overall, I now understand how sessions, authentication, and logout scripts work in PHP.

WEEK 5:DATABASE COMPONENTS

5.1 Database creation

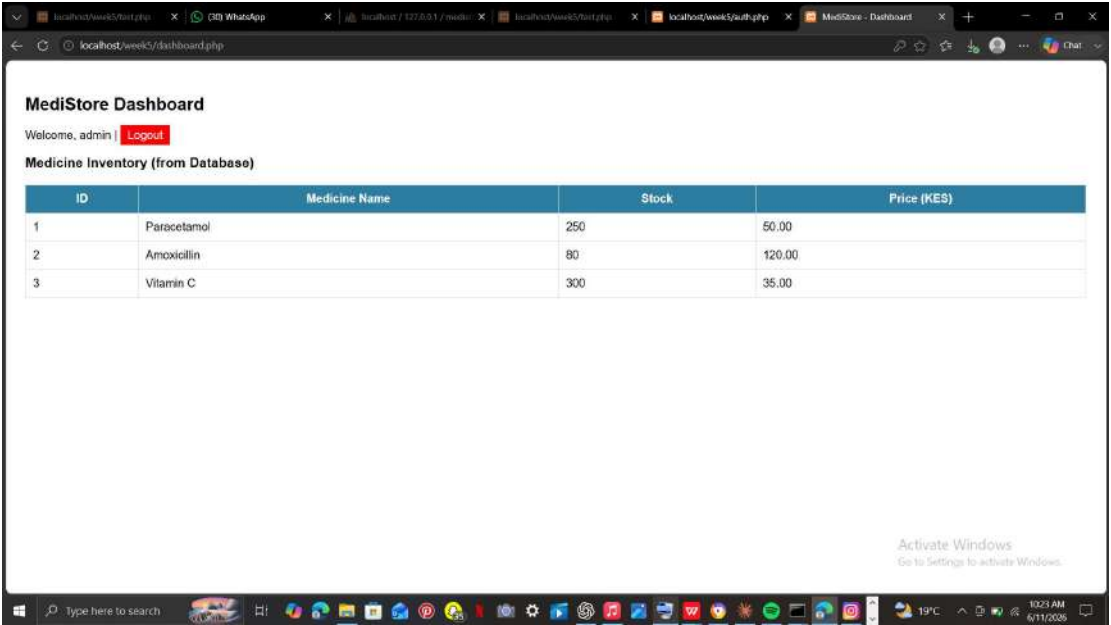


5.2 Login page

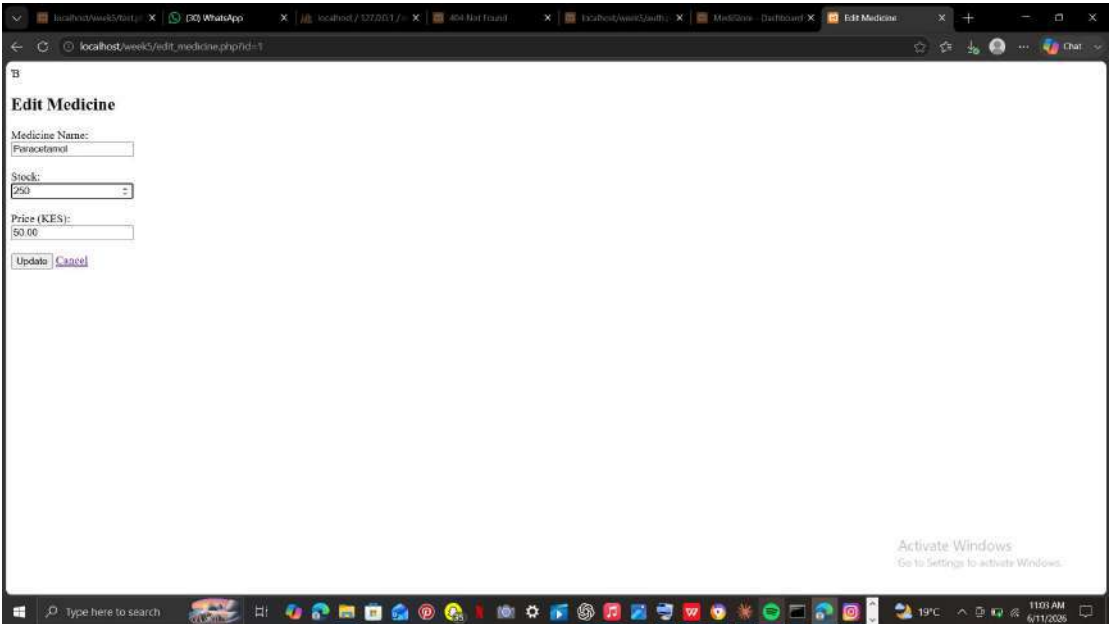


5.3CRUD operations

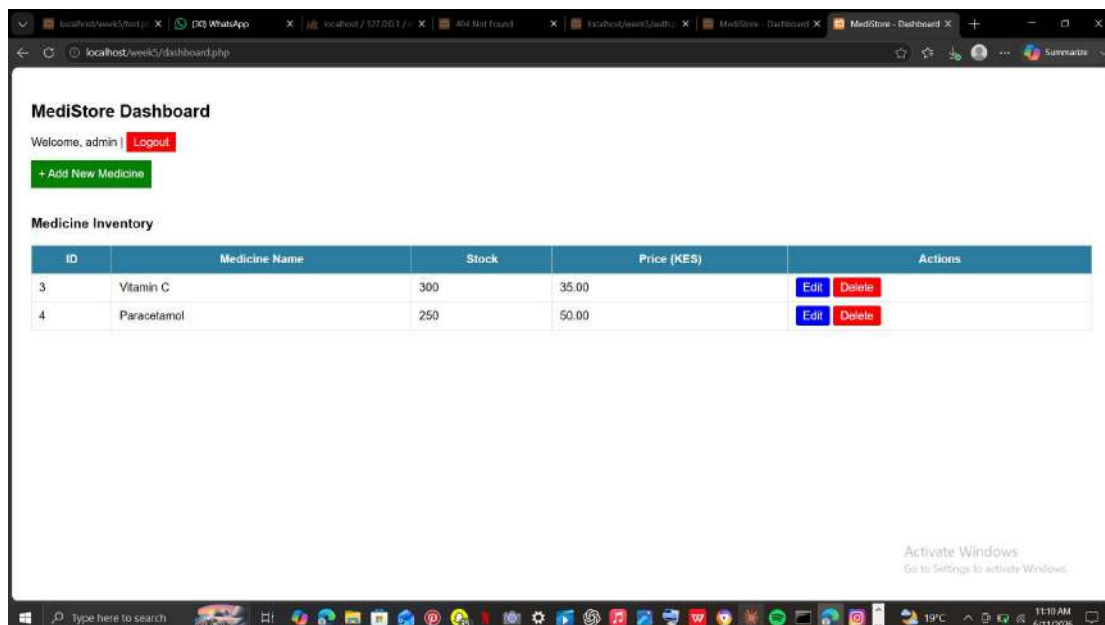
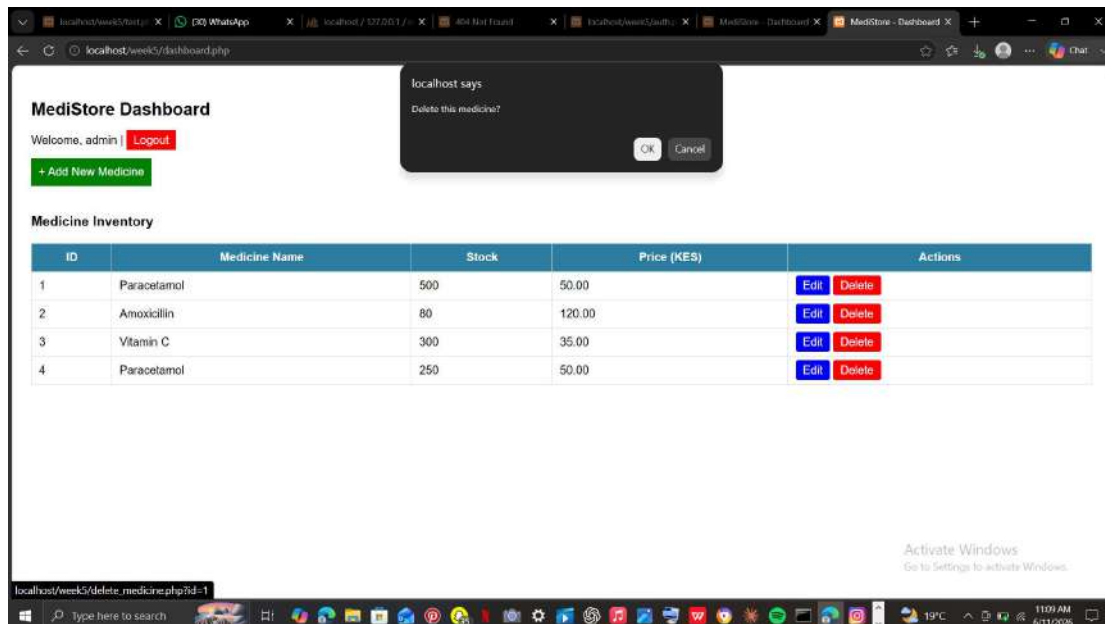
read



Update



Delete



STUDENT REFLECTION

In Week 5, I connected MediStore to a MySQL database called medistore_db with two tables: users and medicines. I implemented all CRUD operations using SQL queries: SELECT to display medicines (Read), INSERT to add new medicines (Create), UPDATE to edit medicine details (Update), and DELETE to remove medicines (Delete). I also added session protection to prevent unauthorized dashboard access. I faced challenges with blank pages and missing buttons, but I fixed them by adding error reporting and updating the code. By the end of Week 5, I successfully built a complete database-driven application where I can Create, Read, Update, and Delete medicine records.

